NIT2112
Object Oriented Programming

Session 03

Inheritance and Polymorphism

VICTORIA UNIVERSITY
MELBOURNE AUSTRALIA

Acknowledgement of Country

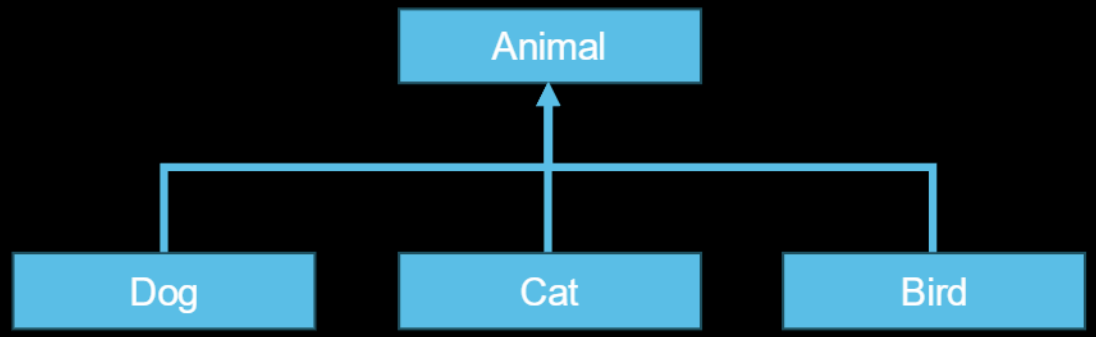
Victoria University acknowledges, recognises and respects the Ancestors, Elders and families of the Bunurong/Boonwurrung, Wadawurrung and Wurundjeri/Woiwurrung of the Kulin who are the traditional owners of University land in Victoria, the Gadigal and Guring-gai of the Eora Nation who are the traditional owners of University land in Sydney, and the Yugara/YUgarapul people and Turrbal people living in Meanjin (Brisbane).

Session Outline

- Inheritance: Concept, Implementation & Types
- Method Overriding & *super()*
- Polymorphism: Concept & Advantages
- Abstract Classes & Methods (*abc* Module)
- Inheritance & Polymorphism Best Practices

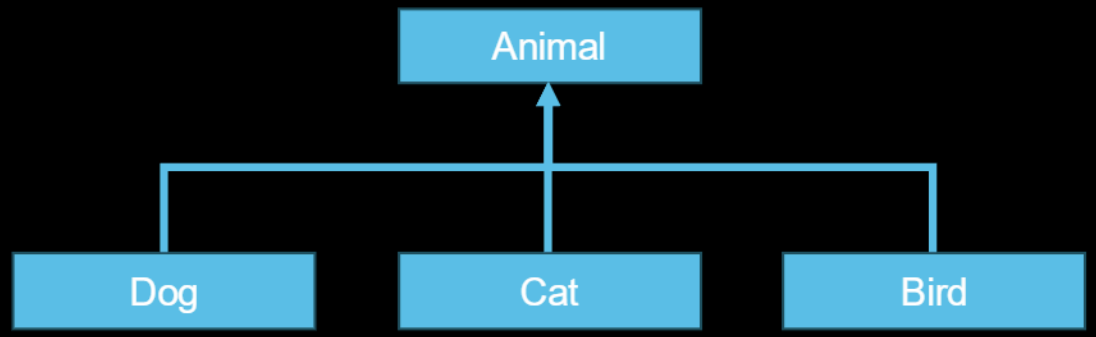
Concepts of Inheritance

- A fundamental OOP concept that enables **code reusability**.
- Allows a new class (subclass/derived class) to **inherit properties and behaviours** from an existing class (superclass/base class).
- Establishes an **"IS-A"** relationship between classes.
- **Reduces** code **duplication** and **enhances maintainability**.



Key Terminology

- Superclass (Base Class/Parent Class): The class whose properties are inherited.
- Subclass (Derived Class/Child Class): The class that inherits properties from the superclass.
- Inheritance: The process of inheriting properties and methods.



Superclass (Base Class/Parent Class)

Inheritance

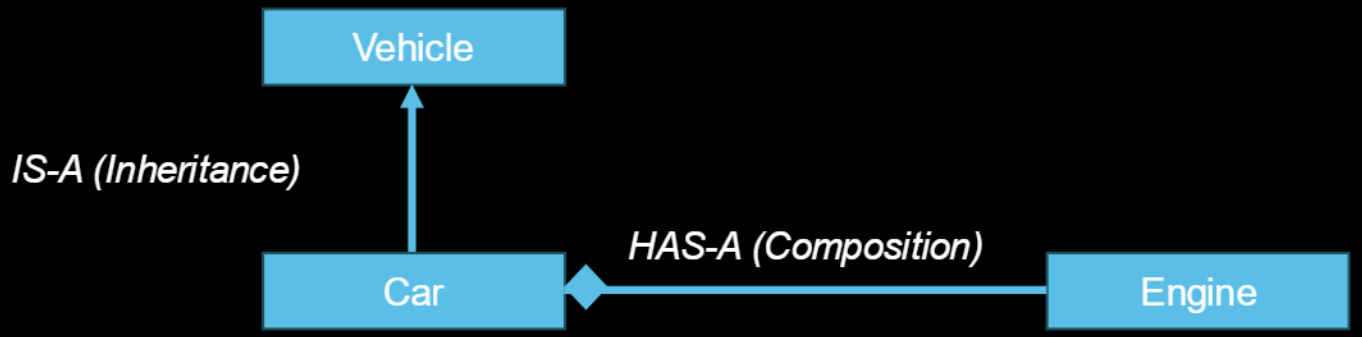
Subclass (Derived Class/Child Class)

Benefits of Inheritance

- **Code Reusability**: Avoid writing the same code multiple times.
- **Improved Maintainability**: Changes in the superclass automatically propagate to subclasses.
- **Extensibility**: Easily add new classes based on existing ones.
- **Abstraction**: Hide complex implementation details from the user.

The "IS-A" Relationship

- Inheritance represents an "**IS-A**" relationship.
- Example: A Car is a Vehicle. A Student is a Person.
- If the "IS-A" relationship doesn't make logical sense, consider using **composition** instead.



Implementing Inheritance in Python

- To inherit from a superclass, include the superclass name in parentheses after the subclass name in the class definition.

```python
class SuperClass:
    # Superclass attributes and methods

class SubClass(SuperClass):
    # Subclass attributes and methods
```

```python
class Vehicle:
    def __init__(self, make, model, year):
        self.make = make
        self.model = model
        self.year = year

    def display_info(self):
        print(f"Vehicle Info: {self.year} {self.make} {self.model}")

class Car(Vehicle):
    def __init__(self, make, model, year, doors):
        super().__init__(make, model, year)
        self.doors = doors

    def display_info(self):
        super().display_info()
        print(f"Car with {self.doors} doors")

my_car = Car("Toyota", "Camry", 2023, 4)
my_car.display_info()
```

Understanding the super() Function

- Used to call methods from the superclass within the subclass.
- *super().__init__()* is used to initialize the superclass's attributes in the subclass's constructor.
- Ensures proper initialization and avoids code duplication.
- Maintains the inheritance chain.

```python
class Vehicle:
    def __init__(self, make, model, year):
        self.make = make
        self.model = model
        self.year = year

    def display_info(self):
        print(f"Vehicle Info: {self.year} {self.make} {self.model}")

class Car(Vehicle):
    def __init__(self, make, model, year, doors):
        super().__init__(make, model, year)
        self.doors = doors

    def display_info(self):
        super().display_info()
        print(f"Car with {self.doors} doors")

my_car = Car("Toyota", "Camry", 2023, 4)
my_car.display_info()
```

A Glimpse at Method Overriding

- The Car class overrides the *display_info()* method from the Vehicle class.
- Allows **subclasses** to provide a **specific implementation** of a method already defined in the superclass.
- We'll cover method overriding in detail later in this session.

```python
class Vehicle:
    def __init__(self, make, model, year):
        self.make = make
        self.model = model
        self.year = year

    def display_info(self):
        print(f"Vehicle Info: {self.year} {self.make} {self.model}")

class Car(Vehicle):
    def __init__(self, make, model, year, doors):
        super().__init__(make, model, year)
        self.doors = doors

    def display_info(self):
        super().display_info()
        print(f"Car with {self.doors} doors")

my_car = Car("Toyota", "Camry", 2023, 4)
my_car.display_info()
```

Code Prediction Challenge

- Question: What will be the output of this code?

```
1   class Animal:
2       def speak(self):
3           print("Generic animal sound")
4
5   class Dog(Animal):
6       def speak(self):
7           print("Woof!")
8
9   animal = Animal()
10  dog = Dog()
11
12  animal.speak()
13  dog.speak()
14
```

Exploring Different Types of Inheritance

- Single Inheritance
- Multiple Inheritance
- Multilevel Inheritance
- Hierarchical Inheritance
- Hybrid Inheritance

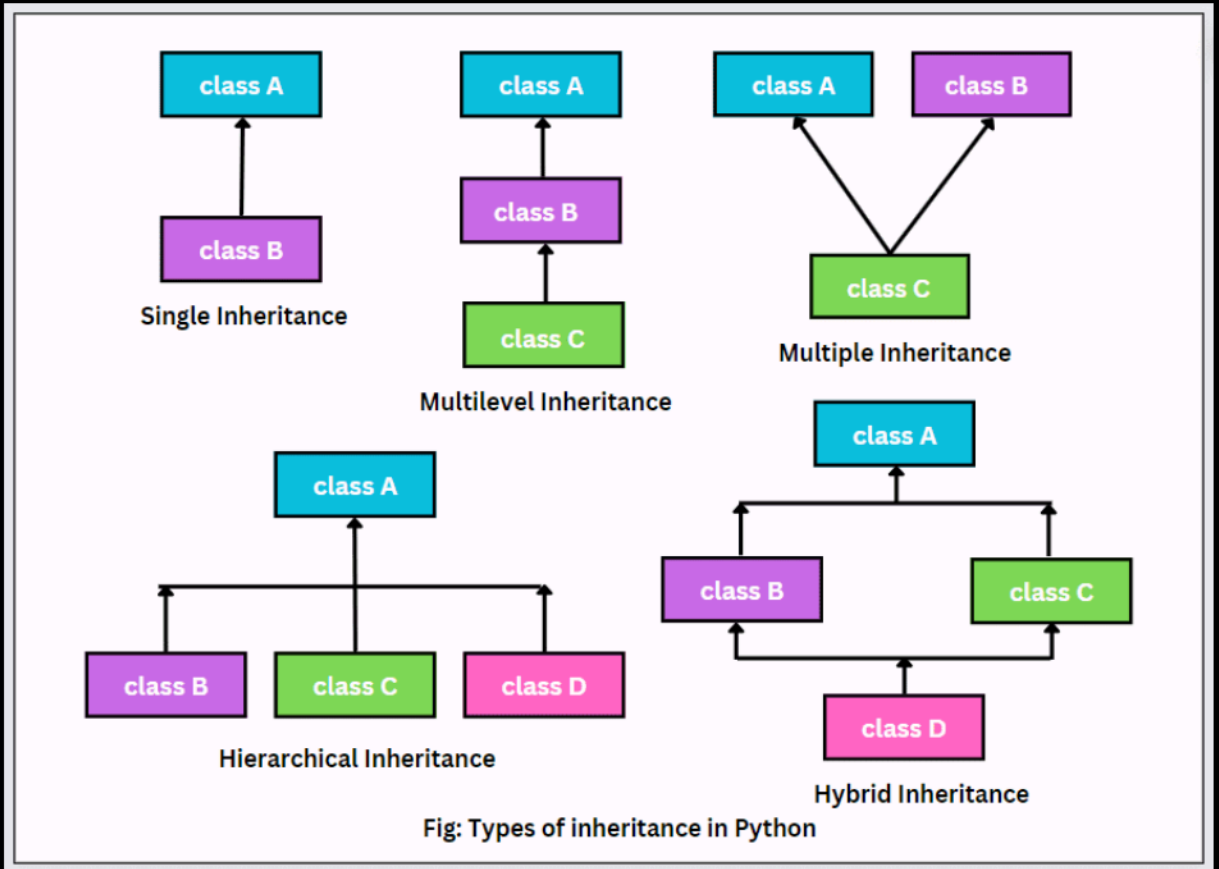
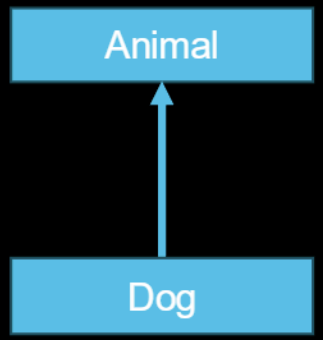


Image from Scientech Easy, March 1, 2025, https://www.scientecheasy.com/2023/09/types-of-inheritance-in-python.html/

Single Inheritance

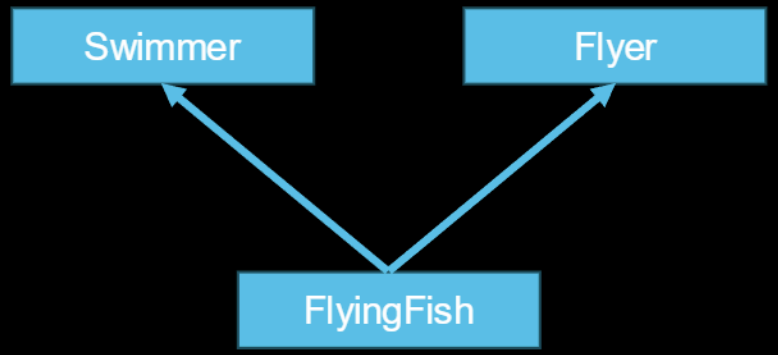
- A subclass inherits from only one superclass.
- Simple and straightforward.
- Good for basic class hierarchies.




```python
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        print("Generic animal sound")

class Dog(Animal):
    def speak(self):
        print("Woof!")

my_dog = Dog("Buddy")
my_dog.speak()  # Output: Woof!
```

Multiple Inheritance

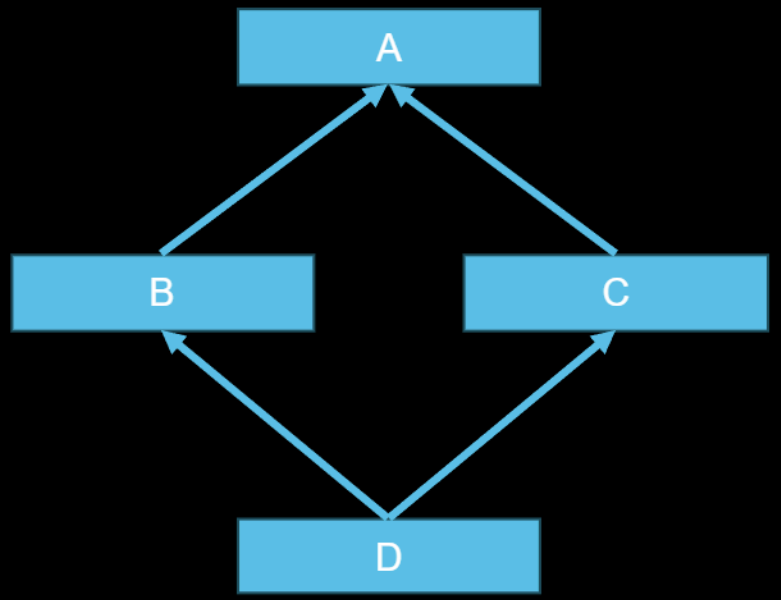
- A class can inherit from multiple superclasses.
- Provides greater flexibility but can lead to complexities (e.g., the "diamond problem").




```python
class Swimmer:
    def swim(self):
        print("Swimming")

class Flyer:
    def fly(self):
        print("Flying")

class FlyingFish(Swimmer, Flyer):
    pass

fish = FlyingFish()
fish.swim()  # Output: Swimming
fish.fly()   # Output: Flying
```

The Diamond Problem

- Occurs when a class inherits from two classes that both inherit from a common superclass.
- Can lead to ambiguity in method resolution.
- Python uses Method Resolution Order (MRO) to handle this.




```python
class Animal:
    def act(self):
        print("General Action")

class Swimmer(Animal):
    def act(self):
        print("Swimming")

class Flyer(Animal):
    def act(self):
        print("Flying")

class FlyingFish(Swimmer, Flyer):
    pass

fish = FlyingFish()
fish.act()  # What is the output?
```

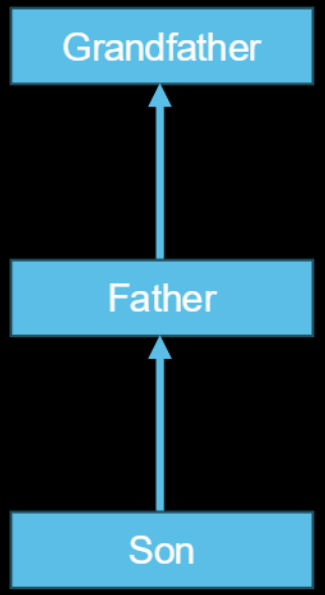
Method Resolution Order (MRO)

- Determines the order in which Python searches for inherited methods.
- Uses the C3 linearization algorithm.
- Can be inspected using *Class.mro()* or *Class.__mro__*.

```python
class Animal:
    def act(self):
        print("General Action")

class Swimmer(Animal):
    def act(self):
        print("Swimming")

class Flyer(Animal):
    def act(self):
        print("Flying")

class FlyingFish(Swimmer, Flyer):
    pass

fish = FlyingFish()
fish.act()  # Output: Swimming

print(FlyingFish.mro())
# Output:
# [<class '__main__.FlyingFish'>, <class '__main__.Swimmer'>,
# <class '__main__.Flyer'>, <class '__main__.Animal'>,
# <class 'object'>]
```

Multilevel Inheritance

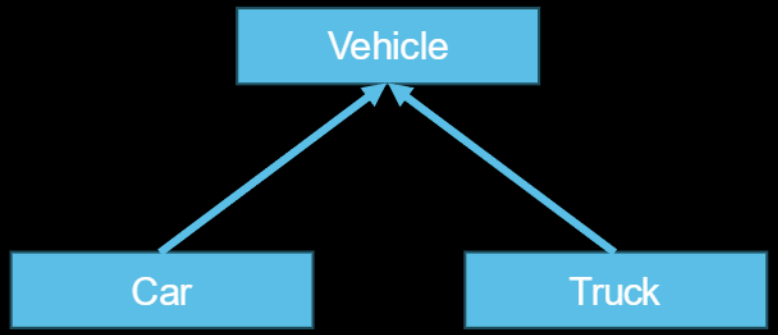
- A class is derived from another class, which is derived from another class.
- Creates a chain of inheritance.




```python
class Grandfather:
    def has_house(self):
        print("Grandfather has a house")

class Father(Grandfather):
    def has_car(self):
        print("Father has a car")

class Son(Father):
    def has_bike(self):
        print("Son has a bike")

son = Son()
son.has_house() # Output: Grandfather has a house
son.has_car()   # Output: Father has a car
son.has_bike()  # Output: Son has a bike
```

Hierarchical Inheritance

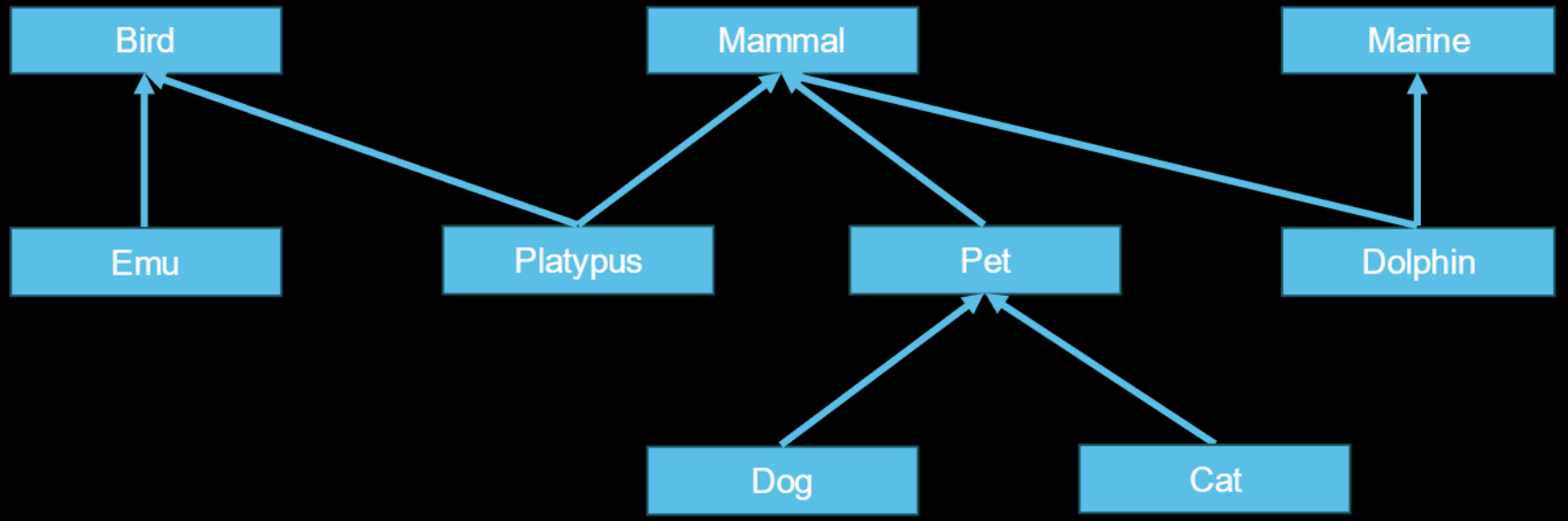
- Multiple subclasses inherit from a single superclass.
- Creates a tree-like inheritance structure.




```python
class Vehicle:
    def __init__(self, make, model):
        self.make = make
        self.model = model

    def display_info(self):
        print(f"Vehicle: {self.make} {self.model}")

class Car(Vehicle):
    def display_info(self):
        super().display_info()
        print("This is a car")

class Truck(Vehicle):
    def display_info(self):
        super().display_info()
        print("This is a truck")

my_car = Car("Toyota", "Camry")
my_truck = Truck("Ford", "F-150")

my_car.display_info()
my_truck.display_info()
```

Hybrid Inheritance

- A combination of different types of inheritance (single, multiple, multilevel, hierarchical).
- Can be complex and requires careful design to avoid ambiguity.



When to Use Which Type of Inheritance

- Single Inheritance: Simple hierarchies, clear "IS-A" relationship.
- Multiple Inheritance: Combining functionalities from different classes (use with caution).
- Multilevel Inheritance: Representing a chain of relationships.
- Hierarchical Inheritance: Multiple specialized classes based on a common base class.
- Hybrid Inheritance: Complex scenarios requiring a combination of inheritance types (use with extreme caution).

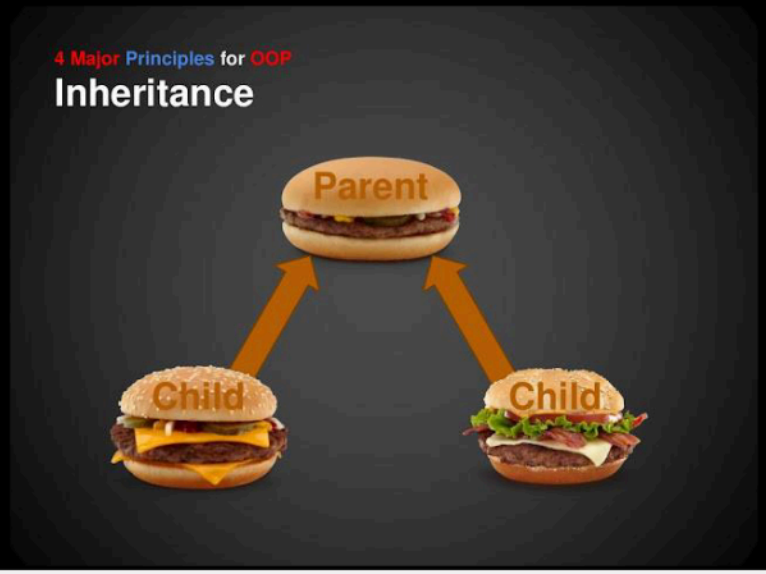


Image from Java67, August 2012, https://www.java67.com/2012/08/what-is-inheritance-in-java-oops-programming-example.html

Method Overriding: Customizing Behaviour

- A subclass provides its **own implementation** of a method already defined in its superclass.
- Allows subclasses to **specialize** inherited behaviour.
- Achieved by defining a method with the **same name, arguments, and return type** in the subclass.

```python
class Animal:
    def speak(self):
        print("Generic animal sound")

class Dog(Animal):
    def speak(self):
        print("Woof!")

class Cat(Animal):
    def speak(self):
        print("Meow!")

animal = Animal()
dog = Dog()
cat = Cat()

animal.speak() # Output: Generic animal sound
dog.speak()    # Output: Woof!
cat.speak()    # Output: Meow!
```

Using super() with Method Overriding

- You can use *super()* to call the superclass's implementation of the overridden method.
- Useful for extending the superclass's behaviour instead of completely replacing it.

```python
class Animal:
    def speak(self):
        print("The animal says:")

class Dog(Animal):
    def speak(self):
        super().speak()  # Call the superclass's speak() method
        print("Woof!")

dog = Dog()

# Output:
#   The animal says:
#   Woof!
dog.speak()
```

Overriding vs. Overloading

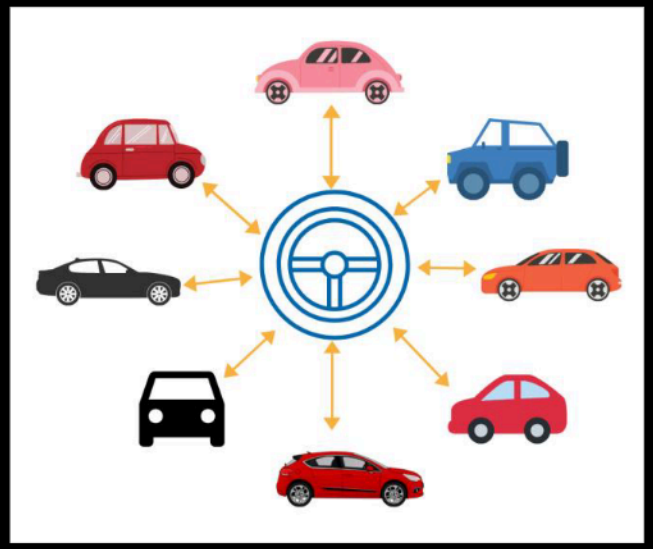
- **Overriding**: Replacing a superclass's method in a subclass (**same name, arguments, and return type**).
- **Overloading**: Defining multiple methods with **the same name but different signatures** in the same class (not directly supported in Python).
- Note: Python doesn't support traditional method overloading like Java or C++. You can achieve similar effects using default arguments or variable arguments (*args, **kwargs).

```python
# "Overloading" Example (using variable arguments)
class Calculator:
    def add(self, a, b, *args):
        total = a + b
        for num in args:
            total += num
        return total


calc = Calculator()

print(calc.add(5, 3))       # Output: 8
print(calc.add(5, 3, 2))    # Output: 10
```

```java
// Overloading Example (Java)
class Calculator {
    public int add(int a, int b) {
        return a + b;
    }

    public int add(int a, int b, int c) {
        return a + b + c;
    }
}

Calculator calc = new Calculator();
System.out.println(calc.add(5, 3));    // Output: 8
System.out.println(calc.add(5, 3, 2)); // Output: 10
```

Polymorphism: Many Forms, One Interface

- The ability of an object to take on many forms.
- Allows objects of **different classes** to be treated as objects of a **common** type.
- Enables writing generic code that can work with different types of objects.



Polymorphism Example

```python
class Animal:
    def speak(self):
        print("Generic animal sound")

class Dog(Animal):
    def speak(self):
        print("Woof!")

class Cat(Animal):
    def speak(self):
        print("Meow!")

def animal_sound(animal):  # Polymorphic function
    animal.speak()

animal = Animal()
dog = Dog()
cat = Cat()

animal_sound(animal) # Output: Generic animal sound
animal_sound(dog)    # Output: Woof!
animal_sound(cat)    # Output: Meow!
```

Duck Typing: "If it walks like a duck..."

- A style of typing in which an object's suitability is determined by the presence of certain methods and properties, rather than its actual type.
- "If it walks like a duck and quacks like a duck, then it must be a duck."
- Python embraces duck typing.

```python
class Duck:
    def quack(self):
        print("Quack!")

class Person:
    def quack(self):
        print("The person imitates a duck: Quack!")

def make_quack(obj):
    obj.quack()

duck = Duck()
person = Person()

make_quack(duck)   # Output: Quack!
make_quack(person) # Output: The person imitates a duck: Quack!
```

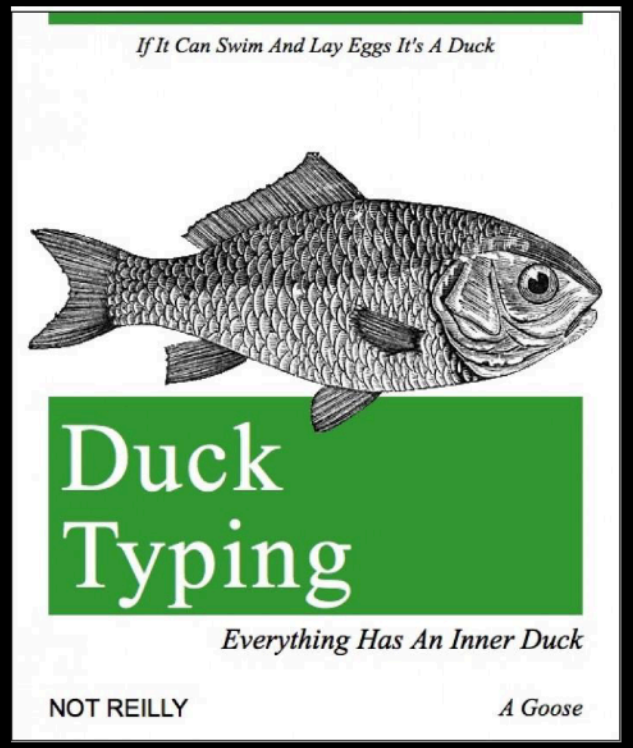


Image from Imgur, April 12 2016, https://imgur.com/duck-typing-O3NKzML

Best Practices for Inheritance and Polymorphism

- Favor composition over inheritance when appropriate.
- Adhere to the Liskov Substitution Principle (LSP).
- Keep inheritance hierarchies shallow.
- Use abstract classes to define clear interfaces.
- Document your code thoroughly.

Composition vs. Inheritance: Making the Right Choice

- Composition:
  - Represents a "**HAS-A**" relationship. One class has a component or instance of another class.
  - Achieves code reuse by combining objects of other classes.
  - Promotes loose coupling, leading to more flexible and maintainable designs.
  - Example: A Car has an Engine. A Computer has a CPU.
- Inheritance:
  - Represents an "**IS-A**" relationship. A subclass is a specialized version of a superclass.
  - Achieves code reuse by inheriting properties and behaviours.
  - Can lead to tight coupling, where changes in the superclass affect subclasses.

```python
# Inheritance Example
class Engine:
    def start(self):
        return "Engine started"

class Car(Engine):  # Car *IS A* Engine (Incorrect!)
    def drive(self):
        return f"{self.start()}, Driving..."

my_car = Car()
print(my_car.drive())  # Output: Engine started, Driving...


# Composition Example
class Engine:
    def start(self):
        return "Engine started"

class Car:  # Car *HAS AN* Engine
    def __init__(self, engine):
        self.engine = engine

    def drive(self):
        return f"{self.engine.start()}, Driving..."

my_engine = Engine()
my_car = Car(my_engine)
print(my_car.drive())  # Output: Engine started, Driving...
```

The Liskov Substitution Principle (LSP): A Key to Polymorphism

- Objects of a superclass should be replaceable with objects of its subclasses **without altering** the correctness of the program.
- Subclasses should **not violate** the contract of their superclasses.

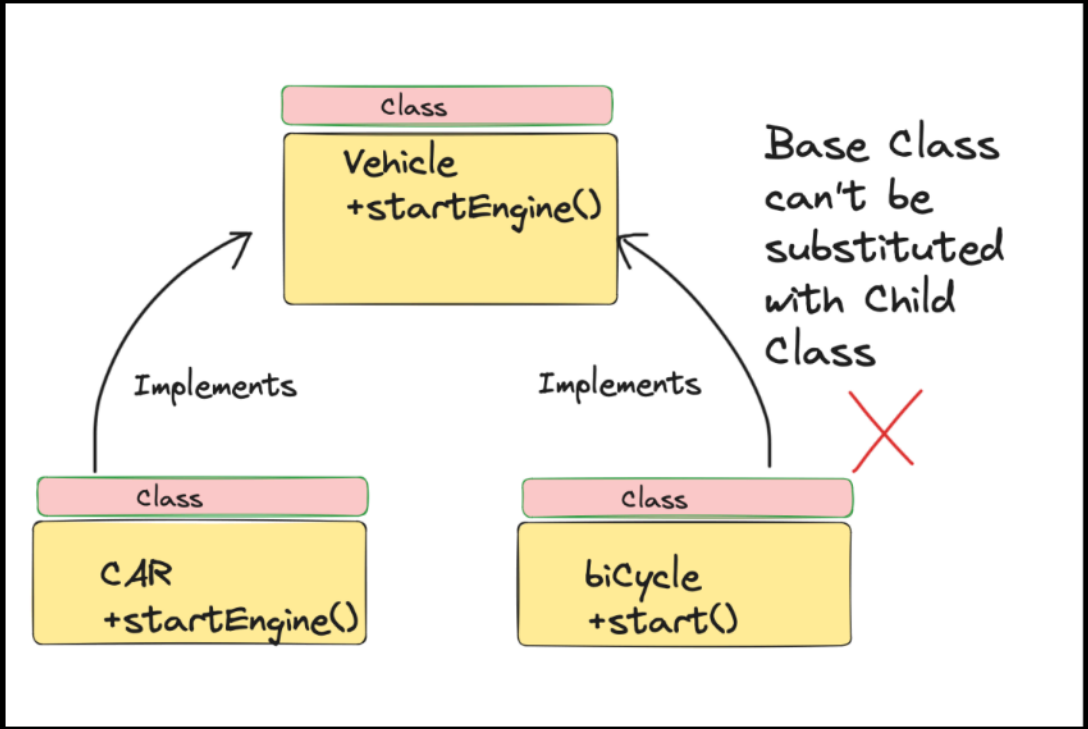


Image from Vikas Taank, Mar 7, 2024, https://medium.com/h7w/understanding-liskov-substitution-principle-lsp-95b4d03c1ca9

The Importance of Documentation

- Document your classes, methods, and inheritance relationships clearly.
- Explain the purpose of each class and method.
- Describe how subclasses extend or override superclass behaviour.
- Good documentation is essential for code maintainability and collaboration.

Visualizing Classes with UML Class Diagrams

- **Unified Modelling Language (UML):** A standardized visual language for specifying, constructing, and documenting the artifacts of software systems.
- **UML Class Diagrams:** A type of UML diagram that describes the structure of a system by showing the classes, their attributes, methods, and the relationships between them.
- Basic Elements:
  - **Class**: Represented as a rectangle divided into three sections:
    - Class name (top section)
    - Attributes (middle section)
    - Methods (bottom section).

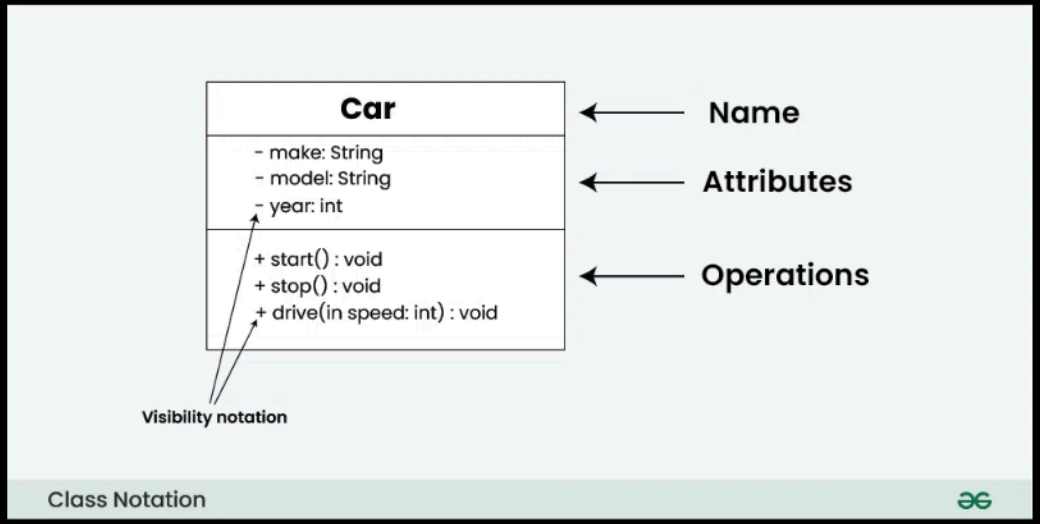
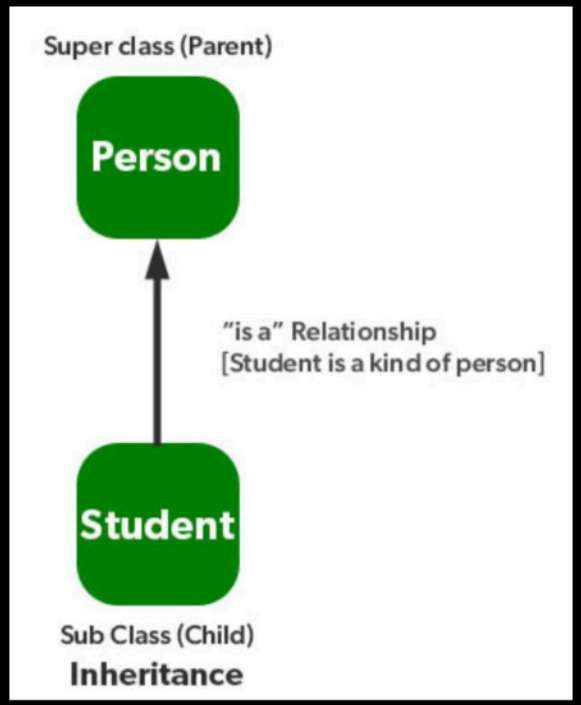


Image from GeeksforGeeks, 03 Jan, 2025, https://www.geeksforgeeks.org/unified-modeling-language-uml-class-diagrams/

Visualizing Classes with UML Class Diagrams

- Basic Elements:
  - **Inheritance**: Represented as a solid line with a hollow triangle pointing towards the superclass.
  - **Composition**: Represented as a solid line with a filled diamond at the aggregate (container) end.



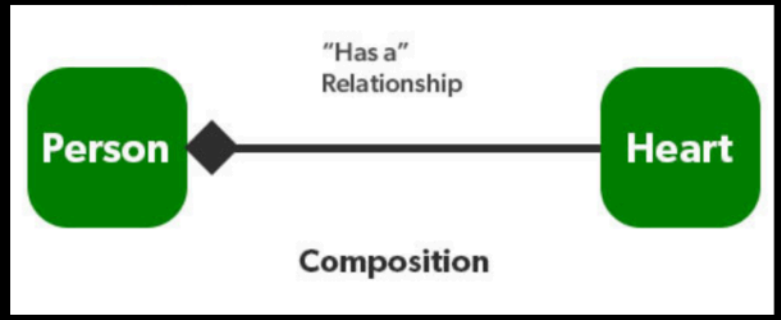


Image from GeeksforGeeks, 17 May, 2024, https://www.geeksforgeeks.org/python-oops-aggregation-and-composition/

Key Takeaways

- Inheritance enables code reusability and establishes "IS-A" relationships.
- Python supports single, multiple, multilevel, hierarchical, and hybrid inheritance.
- Method overriding allows subclasses to customize inherited behavior.
- Polymorphism enables treating objects of different classes in a uniform way.
- Abstract classes define blueprints for subclasses and enforce specific interfaces.
- Follow best practices for effective use of inheritance and polymorphism.

Q & A

VICTORIA UNIVERSITY
MELBOURNE AUSTRALIA

Lab